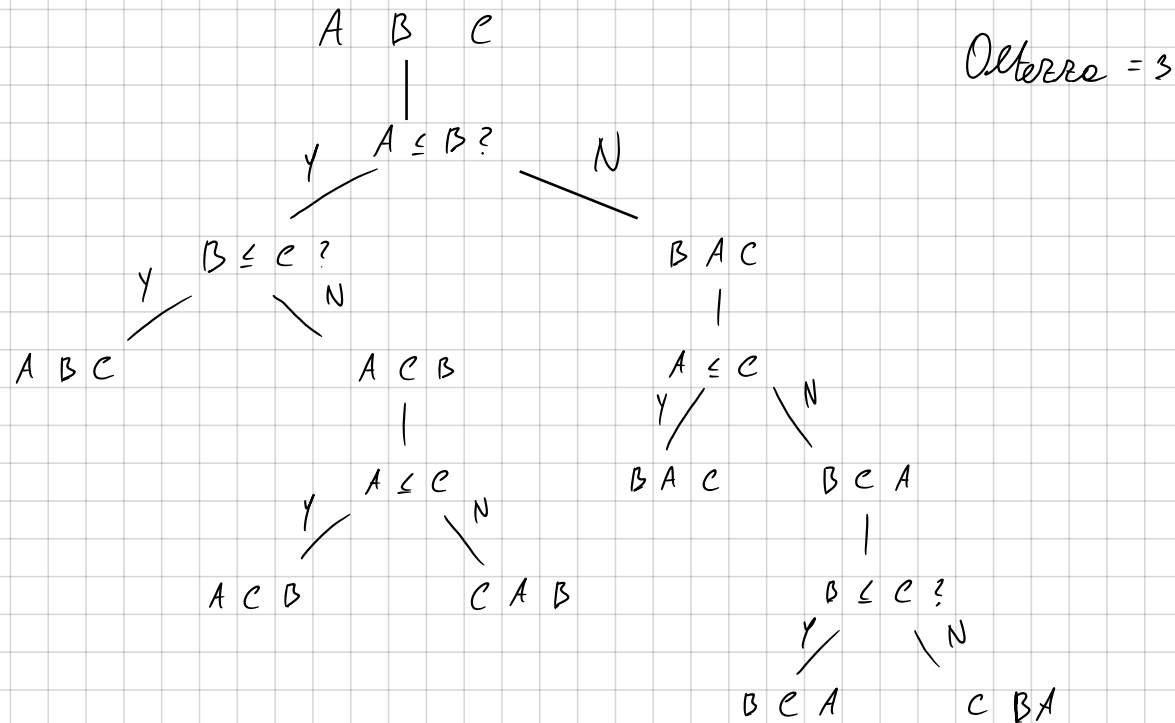


Albero di decisione



Ogni nodo interno fa una domanda che può avere delle risposte fisse. Le foglie sono i risultati finali.

Descrivere completamente il processo di ordinamento effettuato

foglie → permutazione che ordina i valori di partenza

A livello di tempo la lunghezza del percorso è lower bound dei tempi di calcolo.

Albero binario → ogni nodo interno ha 2 figli

A istante il tempo di calcolo è limitato inferiormente dalla lunghezza del percorso (a meno di costanti)

Altezza albero = caso peggiore

Foglie + vicino alla radice = caso migliore

Per capire quanto è alto l'albero devi capire quante foglie devi avere tutte le permutazioni del vettore di input

→ $n^\circ \text{ foglie} \geq n^\circ \text{ permutazioni}$

$n^\circ \text{ flie} \leq 2h$ $h = \text{alterze} \quad \text{also}$

$$\underbrace{n^\circ \text{ permutazioni}} \leq n^\circ \text{ foglie} \leq 2^h$$

$$n! \leq 2^n$$

$$2^k \geq n!$$

$$h \geq \log_2 n$$

$n = \Omega(n \lg n)$ *l'altero* dove n è almeno $n \lg n$

↑
tempo cose regione

lego il suo libro ha almeno $n!$ foglie, con tempo peggiore di gli algoritmo di ordinamento che opera su n elementi.

maybe not $\rightarrow$ è più il migliore, non serve che ne cerchi altri perché sei più che nel caso peggiore non vanno sotto a $\log n$

Counting Sort: algoritmo di ordinamento con $T_{pg} O(n)$

Se fai un confronto che non confronta i valori non puoi fare il ragionamento di prima

Lo **quasi** fore solo per spostamenti di ordinamento per confronto

$\Omega(n \lg n)$

[illegible]

Riuscivisti a capire dove va l'1 nel vettore finale? Sì, perché non ci sono $n + 1$ piccioni

Dove va il 3? 3^a posizione

Dove va il 5? 6^e posizione $\rightarrow$ è il 1° 5 e ce sono 5 n + piccoli di lui

vettoe C $\rightarrow$ vettoe dove conti i n

0	1	2	3	4	5	6	7	8	9

Hai il vettoe con le celle numerate, dentro ci metti quante volte compaiono gli indici corrispondenti

Parti con con tutti 0

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

ed incrementi quando trovi

0	1	2	3	4	5	6	7	8	9
0	2	0	3	0	3	0	1	4	2

copisci che il 5 va in 6^a

posizione perche sommi quello che c'è prima (2 uni e 3 tre), l'ultimo 5 andrà in 8^a posizione (2+3+3)

prefix sum $\rightarrow$ sommi i n ai precedenti. Conti i contenuti delle celle: qual'è la freq dei n $\leq$ e quel n^o

0	1	2	3	4	5	6	7	8	9
0	2	2	5	5	8	8	9	13	15



Indicano la posizione finale dell'ultimo caso in cui appare il n^o (tipo il 3^o tre è il ... 5)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
							5							

Hai preso il 3^o 5, adesso incrementi quanti ne hai

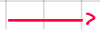
0	1	2	3	4	5	6	7	8	9
0	2	2	5	5	7	8	9	13	15



fai così seguendo l'ordine degli elementi dell'array

Alla fine hai:

0	1	2	3	4	5	6	7	8	9
0	0	2	2	5	5	8	8	9	13



1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	1	3	3	3	5	5	5	7	8	8	8	8	9	9

Pseudo-code

```

function CountingSort(A, K)      CountingSort(A, max(A))
C = vettore di K+1 posizioni 0-based (parte con indice 0)
// esempio C di 0

for i = 0 to K do
    C[i] = 0

for j = 1 to A.length do // incrementi frequenza del val dell'indice
    C[A[j]] = C[A[j]] + 1

for i = 1 to K do // prefix-sum, il 1° elemento è già a posto
    C[i] = C[i] + C[i-1]

B è un vettore di A.length elementi

for j = A.length down to 1 do // su A, va al contrario
    B[C[A[j]]] = A[j]
    C[A[j]] = C[A[j]] - 1

return B
  
```

1° invariante di ciclo $\rightarrow$ In C fino a posizione $i-1$ son tutti 0

le altre invarianti di ciclo $\rightarrow$ alla fine del 1° ciclo ha la frequenza del valore fino a $j-1$

Tempo

1° for $\rightarrow \Theta(K)$

2° for $\rightarrow \Theta(n)$

3° for $\rightarrow \Theta(K)$

4° for $\rightarrow \Theta(n)$ 2 operazioni per n volte

tempo totale: $\Theta(K) + \Theta(n) + \Theta(n) + \Theta(n) = \Theta(n+K) = T(n, K)$ questo tempo è $O(n)$ se $K = O(n)$ (perché diventa $O(n+n)$)

ci dice quando ha senso il counting sort

Se hai una sola cifra tipo:

$A = 3 \quad 1000000 \quad 1$ ora in C l'ultimo $C = \begin{array}{c} 0 \quad 1 \\ 0 \quad 1 \quad \dots \quad \dots \quad \dots \end{array}$

In place? $\rightarrow$ no, se C e B , in certi casi posso fare e meno di B , ma non C

Stabile? $\rightarrow$ si

Se counting sort viene usato come procedura di ordinamento di un altro algoritmo, il **Radix Sort**, si usano più vettori con elementi con cifre limitate

function **Radix Sort** (A, d)
 $\leftarrow$ n° di cifre massimo di ciascun valore di A
 for $i = 1$ to d do
 ordina A in base alla i -esima cifra con un algoritmo di ordinamento stabile

$A = 411 \quad 850 \quad 150 \quad 917$

$A = \begin{Bmatrix} 4 & 1 & 1 \\ 8 & 5 & 0 \\ 1 & 5 & 0 \\ 4 & 1 & 7 \end{Bmatrix}$

Tempo radix sort $\rightarrow O(d \cdot \text{tempo ordinamento intero})$

Se usi il counting sort $O(d \cdot n)$

$A = \begin{Bmatrix} 8 & 5 & 0 \\ 1 & 5 & 0 \\ 4 & 1 & 1 \\ 4 & 1 & 7 \end{Bmatrix}$

$A = \begin{Bmatrix} 4 & 1 & 1 \\ 4 & 1 & 7 \\ 8 & 5 & 0 \\ 1 & 5 & 0 \end{Bmatrix}$

$A = \begin{Bmatrix} 1 & 5 & 0 \\ 4 & 1 & 1 \\ 4 & 1 & 7 \\ 8 & 5 & 0 \end{Bmatrix}$